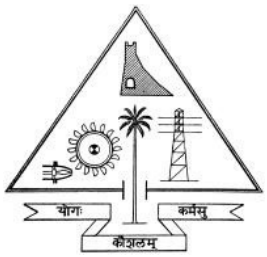
INTELLIGENT PERSONAL DATA LEAKAGE DETECTION SYSTEM USING DEEP LEARNING AND RULE-BASED ANALYSIS

Mini Project Report submitted in partial fulfillment of the requirements for the award of the degree of Master of Computer Applications of the APJ Abdul Kalam Technological University

submitted by

ABRAR NASSER

(TCR25MCA-2003)



DEPARTMENT OF COMPUTER APPLICATIONS
GOVERNMENT ENGINEERING COLLEGE

THRISSUR - 680009

JULY 2026

DECLARATION

I hereby declare that the mini project named, INTELLIGENT PERSONAL DATA LEAKAGE DETECTION SYSTEM USING DEEP LEARNING AND RULE-BASED ANALYSIS, is my own work and that, to the best of my knowledge and belief, it contains no material previously published by another person nor material which has been accepted for the award of any other degree or course of the university or any other institute of higher learning, except where due acknowledgment and reference has been made in the text.

Place : Thrissur

Date : 15-07-2026

ABRAR NASSER
(TCR25MCA-2003)

ACKNOWLEDGEMENT

I would like to thank Department of Computer Application, GEC Thrissur, for giving me this opportunity to pursue this project and successfully complete it.

I am highly indebted to our head of the department Dr. Sminesh C N, for his guidance and constant supervision as well as providing necessary information regarding the project and also for his support in completing the project.

I express my heartfelt gratitude to my project guide Prof. Sankaranarayanan P N, for his committed guidance, valuable suggestions, constructive criticisms and precious time that he invested throughout the work.

I express my heartfelt gratitude to project coordinator Prof. Raseek C, for his committed guidance as well as providing necessary information regarding the project, and also for his support in completing the project.

I sincerely thank all other faculties of MCA department for guiding me through the processes involved in the project.

ABSTRACT

The rapid growth of digital communication platforms such as emails, cloud storage, messaging applications and enterprise systems has significantly increased the risk of personal data leakage. Sensitive information such as passwords, financial data, Personally Identifiable Information (PII) and confidential organizational data is often unintentionally or maliciously exposed. Existing systems face challenges such as a lack of contextual understanding, an inability to detect unstructured leaks and high false-positive rates, since rule-based and signature-based systems detect only predefined patterns and lack context awareness, while conventional machine learning models lack deep contextual understanding and encryption and access control mechanisms fail to prevent insider threats.

To address these challenges, this project proposes an Intelligent Personal Data Leakage Detection System that combines rule-based filtering for known, structured patterns with a deep learning and Natural Language Processing (NLP) approach. The system integrates transformer-based NLP contextual analysis, deep learning classification using LSTM/CNN models, rule-based filtering, anomaly detection for insider threats, real-time monitoring and alerts, and an explainable AI layer, so that content-level detection and behavioural insight are combined within a single pipeline.

The proposed system is expected to provide real-time detection of personal data leaks with reduced false positives, generate alerts along with a computed risk score and present explanations and reports so that every decision can be understood and acted upon by a security analyst. The system is evaluated using standard parameters such as accuracy, precision, recall, F1-score, false positive rate, detection latency and throughput. Overall, this project demonstrates how combining rule-based precision, deep-learning-based contextual understanding and behavioural anomaly detection can result in a data leakage detection system that is accurate, interpretable and suitable for near real-time enterprise deployment.

CONTENTS

List of Figures viii
List of Abbreviations ix
1 INTRODUCTION 1
2 LITERATURE REVIEW 4
2.1 Deep Learning-Based Sensitive Data Leakage Detection in Cloud Environments 4
2.2 AI-Driven Insider Threat Detection for Data Leakage Prevention 5
2.3 Natural Language Processing Approach for Personal Data Leakage Detection 5
2.4 Hybrid Approach for Real-Time Data Leakage Detection 6
2.5 Comparative Summary of Literature Reviewed 6
2.6 Summary and Research Gap 6
3 ENVIRONMENTAL STUDY 9
3.1 System Configuration 9
3.2 Software Specification 12
4 SYSTEM ANALYSIS 15
4.1 Requirements Analysis 15
4.2 Existing System 16
4.3 Limitations of Existing System 16
4.4 Proposed System 17
4.5 Advantages of Proposed System 17
4.6 Functional Requirements 18
4.7 Non-Functional Requirements 20
4.8 Feasibility Study 21
Bibliography 23

LIST OF FIGURES

LIST OF ABBREVIATIONS

AI	–	Artificial Intelligence
BERT	–	Bidirectional Encoder Representations from Transformers
CNN	–	Convolutional Neural Network
DLP	–	Data Leakage Prevention
FPR	–	False Positive Rate
GPU	–	Graphics Processing Unit
IDE	–	Integrated Development Environment
IEEE	–	Institute of Electrical and Electronics Engineers
LIME	–	Local Interpretable Model-agnostic Explanations
LSTM	–	Long Short-Term Memory
NLP	–	Natural Language Processing
PDF	–	Portable Document Format
PII	–	Personally Identifiable Information
SHAP	–	SHapley Additive exPlanations
SRS	–	Software Requirements Specification
VS Code	–	Visual Studio Code
XAI	–	Explainable Artificial Intelligence

CHAPTER 1
INTRODUCTION

The rapid growth of digital communication platforms such as email, cloud storage and enterprise information systems has made the accidental or malicious exposure of sensitive personal information a persistent and growing concern. Data such as passwords, financial details and other Personally Identifiable Information (PII) is routinely handled by individuals and organisations who may not realise, at the moment of sharing a document or a message, that it contains information capable of causing real harm if exposed. Data Leakage Prevention (DLP) has therefore become a central concern within cybersecurity, motivating a substantial body of work on how leakage can be detected automatically before it causes damage.

Traditional security mechanisms such as encryption and access control are effective at restricting who can obtain data in the first place, but they offer no protection once legitimate access has already been granted. An authorised user, whether acting carelessly or maliciously, can still copy, forward or upload sensitive content through channels that encryption and access control were never designed to police. This observation has motivated a distinct line of research into content-aware Data Leakage Prevention, in which systems inspect the content of outgoing data itself, rather than only the identity of the party accessing it, to decide whether that content should be permitted to leave a trusted boundary.

Early content-aware DLP systems relied almost entirely on rule-based pattern matching: regular expressions and keyword lists tuned to recognise structured identifiers such as email addresses, phone numbers or credit card numbers. These systems remain fast and precise for the exact patterns they are configured to recognise, but they possess no understanding of context and are easily defeated by paraphrasing, formatting variation or any leakage pattern that was not anticipated in advance. Signature-based detection extends this idea to known threat indicators but is similarly blind to novel or evolving leakage patterns. Traditional machine learning approaches improved on this by learning statistical regularities from labelled examples, but most such approaches still struggle with unstructured, free-form text and cannot easily separate genuinely sensitive content from superficially similar but harmless content.

More recent work has approached the problem using deep learning and transformer-based Natural Language Processing (NLP), which can capture the semantic context of a passage of text rather than only its surface form, and using anomaly detection, which looks for abnormal patterns of data access or transmission that may indicate an insider threat rather than a single sensitive string. Despite this progress, most deployed and proposed systems still rely predominantly on a single detection paradigm, either rule-based filtering or a purely learned model, rather than combining the precision of rule-based pattern matching with the contextual

understanding of deep learning and NLP and the behavioural insight of anomaly detection. In addition, many learned detection systems operate as black boxes, producing a classification without an accompanying explanation, which limits their usefulness to security analysts who must justify and act on each detection decision.

This project addresses that gap by proposing an Intelligent Personal Data Leakage Detection System that combines deep learning and rule-based analysis. Rather than relying on a single detection technique, the proposed system integrates rule-based filtering for known, structured patterns; transformer-based NLP for contextual, semantic-level analysis; deep learning models (LSTM/CNN) for detecting complex or previously unknown leakage patterns; anomaly detection for identifying insider-threat behaviour; and an explainable AI layer that produces a human-readable rationale for every detection decision. Together, these components are intended to provide real-time, context-aware detection of sensitive data leakage while keeping the false-positive rate low and the system's decisions interpretable to the analysts who rely on it.

The increasing volume and sensitivity of data handled by enterprise communication and storage platforms highlights the limitations of relying on a single detection paradigm. A system that combines the speed and precision of rule-based matching with the contextual depth of deep learning and NLP, the behavioural insight of anomaly detection and the transparency of explainable AI is better positioned to detect both known and previously unseen leakage patterns while remaining trustworthy to the analysts who act on its output. Therefore, reviewing the existing literature is essential to identify current approaches, their limitations and the potential areas for improvement, which are further explored in the following chapter.

CHAPTER 2
LITERATURE REVIEW

Data leakage detection and prevention has been studied extensively within venues sponsored by the Institute of Electrical and Electronics Engineers (IEEE) and other peer-reviewed publishers, spanning deep-learning-based content inspection, AI-driven insider-threat detection, NLP-based semantic analysis and hybrid real-time architectures. This chapter reviews four representative, recent works that together establish the algorithmic foundation for the proposed Intelligent Personal Data Leakage Detection System.

2.1 Deep Learning-Based Sensitive Data Leakage Detection in Cloud Environments

A foundational requirement for content-aware leakage detection in modern enterprise settings is the ability to analyse large volumes of data moving through cloud infrastructure. Guan and Chen address this by applying deep learning techniques to the detection and protection of sensitive data leakage, quantitatively evaluating leakage risk and reporting the accuracy and recall of the resulting detector [1]. Their approach moves beyond static, signature-based scanning of cloud-stored content by using learned representations to identify sensitive information even when it does not exactly match a predefined pattern. This work demonstrates the feasibility of deep-learning-driven content inspection at cloud scale, but it is oriented primarily toward detecting sensitive content itself, rather than jointly reasoning about the context surrounding that content or the behaviour of the user handling it.

2.2 AI-Driven Insider Threat Detection for Data Leakage Prevention

Content inspection alone cannot capture leakage that results from the actions of an authorised user rather than from the presence of a recognisable sensitive pattern. Alzaabi and Mehmood address this complementary problem through a comprehensive review of machine-learning-based malicious insider-threat detection, consolidating recent advances, open challenges and opportunities across the field [2]. Their work surveys techniques for modelling and flagging anomalous user behaviour that may indicate an impending or ongoing leakage incident, extending DLP beyond simple content matching into the domain of behavioural analysis. While informative for identifying abnormal access or transmission patterns, this line of work is generally studied separately from content-level detection, leaving open the question of how behavioural and content-based signals can be combined within a single system.

2.3 Natural Language Processing Approach for Personal Data Leakage Detection

Structured identifiers such as email addresses and phone numbers can be captured reliably through pattern matching, but unstructured, free-form text often expresses sensitive personal information without following any fixed syntactic pattern. Mainetti and Elia address this limitation by proposing a BERT-based NLP approach to personal data leakage detection [3], using language-understanding techniques to identify sensitive personal information based on its semantic content rather than its surface form alone. This contextual approach is directly relevant to the proposed system's NLP-based contextual analysis component, though the reviewed work concentrates on content-level detection and does not integrate rule-based filtering or anomaly-based behavioural signals alongside its NLP analysis.

2.4 Hybrid Approach for Real-Time Data Leakage Detection

Combining multiple detection techniques within a single, real-time pipeline is essential for practical deployment, since no single technique is equally well suited to every type of leakage. Santhosh et al. propose an enhanced data leakage detection approach for cloud and enterprise environments that combines NLP-driven content classification, metadata tagging and anomaly-based learning [4], improving detection responsiveness and coverage over single-technique approaches. This work most closely anticipates the hybrid philosophy of the proposed system, demonstrating that combining textual and behavioural signals can improve real-time detection performance, though it does not incorporate an explicit rule-based filtering stage for known structured identifiers or a dedicated explainability layer to make its detection decisions transparent to a human analyst.

2.5 Comparative Summary of Literature Reviewed

Table 2.1 presents a comparative summary of the four works reviewed in this chapter, highlighting the focus area, the specific AI application to data leakage detection, the key contribution of each work and the challenges or limitations that motivate the hybrid design of the proposed system.

Author(s) & Year	Focus Area	AI Application in Leakage Detection	Key Contribution	Challenges / Limitations
Guan and Chen (2024) [1]	Cloud Data Protection	Deep-learning content inspection of cloud-stored data	Detects sensitive content without an exact pattern match	Content-focused; no behavioural context
Alzaabi and Mehmood (2024) [2]	Insider Threat Detection	ML-based review of behavioural anomaly detection	Consolidates insider-threat ML techniques and open issues	Behavioural focus only; not linked to content signals

Author(s) & Year	Focus Area	AI Application in Leakage Detection	Key Contribution	Challenges / Limitations
Mainetti and Elia (2025) [3]	NLP-Based Detection	BERT-based semantic PII identification	Detects PII from unstructured text via context	Content-level only; no rule or anomaly layer
Santhosh et al. (2025) [4]	Hybrid Real-Time DLP	NLP classification with metadata tagging and anomaly learning	Combines textual and behavioural signals for leakage	No explicit rule-based stage; limited explainability

2.6 Summary and Research Gap

Across the reviewed literature, three recurring limitations emerge. First, deep-learning-based content inspection approaches, such as the cloud-oriented detector proposed by Guan and Chen [1], are effective at identifying sensitive content directly but are not designed to reason about user behaviour or insider-threat indicators. Second, behavioural and NLP-based approaches, represented respectively by Alzaabi and Mehmood's insider-threat review [2] and Mainetti and Elia's NLP-based leakage detector [3], each address an important dimension of the leakage-detection problem largely in isolation, but neither integrates the other's signal, nor a rule-based layer for efficiently handling known structured patterns. Third, while Santhosh et al.'s hybrid NLP-and-anomaly model [4] shows that combining learning components improves detection performance, and recent explainable-AI work such as Gaspar et al.'s LIME/SHAP-based intrusion-detection study [6] shows that model decisions can be made interpretable to analysts, none of the reviewed systems combine rule-based filtering, contextual NLP analysis, deep-learning-based pattern detection, behavioural anomaly detection and an explainable AI layer within a single, unified architecture. Recent deep-learning-based DLP frameworks, such as that of Nikam et al. [5], likewise emphasise detection accuracy without a comparable focus on explainability or a rule-based fast path for known patterns.

The proposed Intelligent Personal Data Leakage Detection System directly addresses this gap by integrating all four detection paradigms, rule-based filtering, NLP-based contextual analysis, deep learning pattern detection and anomaly-based insider-threat detection, and by adding an explainable AI layer that makes each detection decision interpretable, building on the individual strengths established in the reviewed literature while addressing the fragmentation between them. Having identified this gap, the next chapter describes the environmental study, covering the hardware and software resources required to build and evaluate the proposed system.

CHAPTER 3
ENVIRONMENTAL STUDY

This chapter discusses the environmental settings required to successfully develop, implement and evaluate the Intelligent Personal Data Leakage Detection System. The environment of a project is important because it ensures that every component of the system, the rule-based filter, the NLP-based contextual analyser, the deep learning pattern detector, the anomaly detection module, the risk-scoring engine and the explainable AI reporting layer, performs its function correctly and integrates smoothly with the others.

The environment entails both the hardware and software pre-requisites required for the system to operate successfully under realistic conditions. It covers the tools, frameworks and platforms used throughout development, since the correct configuration of these elements directly affects the training time, inference latency and reliability of the detection pipeline.

3.1 System Configuration

System configuration refers to the collection of hardware and software components necessary to develop and execute the system. The proposed system performs several computationally distinct operations, rule-based pattern matching, transformer-based semantic analysis, deep learning inference, behavioural anomaly detection and explanation generation, so a properly configured environment is essential to keep processing time reasonable during both model training and near real-time monitoring.

3.1.1 Hardware Requirements

The hardware requirements outline the minimum resources necessary to train the deep learning and NLP models, run the rule-based and anomaly detection components and serve near real-time alerts without noticeable delay. The key hardware requirements are detailed as follows:

- Processor: Intel® Core™ i5 Processor / AMD Ryzen™ 5 Processor or higher, to handle rule-based filtering, text preprocessing and orchestration of the detection pipeline efficiently.
- Memory: Minimum 8 GB RAM (16 GB recommended), to support batch processing of text and document inputs alongside simultaneous execution of the NLP, deep learning and anomaly detection components.
- GPU (optional but recommended): A CUDA-capable GPU with at least 4 GB of video memory, to accelerate training and inference of the transformer-based (BERT) and LSTM/CNN-based deep learning models. The system remains usable on CPU-only hardware for smaller-scale inference, at the cost of higher detection latency.

- Storage: 20–50 GB of free disk space, for storing training corpora, pre-trained model checkpoints, detection logs and generated explanation reports.

3.1.2 Software Requirements

The software requirements specify the platforms, languages and libraries used to implement rule-based filtering, NLP-based contextual analysis, deep learning pattern detection, anomaly detection, risk scoring and explainable reporting. Ensuring compatibility between these tools is important because the individual detection components must exchange data in a consistent format. The key software requirements are detailed as follows:

- Operating System: Windows 10, Ubuntu 20.04 LTS or macOS, to support cross-platform development and execution.
- Programming Language: Python 3.10 or higher, used for implementing the rule-based filter, the NLP and deep learning components, the anomaly detector and the supporting pipeline.
- Version Control System: Git, for tracking changes to the codebase, model configurations and detection-rule definitions.
- PyTorch – a deep-learning framework used to implement and train the transformer-based NLP model and the LSTM/CNN-based pattern-detection models.
- Hugging Face Transformers – used to load and fine-tune a pre-trained BERT model for contextual, semantic-level analysis.
- spaCy – used for text preprocessing and lightweight NLP support alongside the transformer-based component.
- scikit-learn – used to implement and evaluate the anomaly detection component and to compute evaluation metrics such as precision, recall and F1-score.
- Regex (Python's built-in re module) – used to implement the rule-based filtering component for known patterns such as email addresses and phone numbers.
- Flask (or an equivalent lightweight web framework) – used to serve the monitoring dashboard through which alerts, risk scores and explanation reports are presented.
- SHAP / LIME – used to implement the explainable AI layer, generating human-readable rationales for detection decisions.

3.1.3 Software Requirements for Development

Besides the execution specifications, certain tools are indispensable for development, testing and maintenance of the system. These tools assist in writing and debugging the detection

pipeline, managing the labelled calibration dataset and tracking experiments across model versions. The key development software requirements are detailed as follows:

- Operating System: Windows 10, Ubuntu 20.04 LTS or macOS, ensuring cross-platform compatibility.
- Back End: Python 3.10 or later, serving as the core implementation language for every module of the system.
- IDE Used: Visual Studio Code, used for development, debugging and experiment management.
- Version Control: Git, for managing source code, model configurations and the labelled calibration dataset.
- PyTorch and Hugging Face Transformers – for training and fine-tuning the NLP and deep learning components.
- Jupyter Notebook – for exploratory analysis of detection results and iterative tuning of the risk-scoring mechanism.

3.2 Software Specification

This section describes the individual software components and technologies used to build the system in more detail. Each component performs a specific role within the pipeline, filtering, contextual analysis, pattern detection, anomaly detection, risk scoring or explanation, and the combination of these tools is intended to ensure a smooth, reproducible development process.

3.2.1 Python 3.10

Python was chosen as the primary programming language for this project due to its readability, its extensive ecosystem of machine-learning and NLP libraries and the strong community support available for both deep learning and cybersecurity research. Rule-based filtering, contextual analysis, deep learning inference, anomaly detection and the monitoring dashboard are all implemented in Python, allowing the different modules of the pipeline to interoperate without cross-language data conversion overhead.

3.2.2 Visual Studio Code

Visual Studio Code (VS Code) was selected as the primary Integrated Development Environment (IDE) used during development, testing and maintenance of the system. It offers strong Python support, integrated debugging, Git integration and extensions for Jupyter notebooks, which together streamline the iterative process of tuning the detection models and the risk-scoring mechanism.

3.2.3 PyTorch and Transformer-Based NLP (BERT)

PyTorch is used to implement the deep learning components at the core of the system, including the LSTM/CNN-based pattern-detection models. A pre-trained transformer-based model such as BERT, loaded and fine-tuned through the Hugging Face Transformers library, is used to perform the contextual, semantic-level analysis that distinguishes genuinely sensitive content from superficially similar but non-sensitive text.

3.2.4 Rule-Based Filtering (Regex)

Python's built-in regular-expression capabilities are used to implement the rule-based filtering component, which identifies known, structured patterns such as email addresses and phone numbers efficiently and with high precision, complementing the contextual analysis performed by the NLP and deep learning components.

3.2.5 Anomaly Detection

An anomaly detection module, implemented using scikit-learn, is used to identify data-handling behaviour that deviates from established normal patterns, supporting the system's insider-threat detection capability alongside its content-based detection components.

3.2.6 Flask

Flask is used to serve the monitoring dashboard, a lightweight web interface through which security analysts view detected sensitive data, risk scores, alerts and the corresponding explanation reports. Flask's minimal footprint keeps this interface simple to develop and to demonstrate without introducing unnecessary infrastructure.

3.2.7 Git

Git is used as the version control system for the project's source code, model configuration files and the labelled calibration dataset used to tune the detection components. Version control is particularly important for this project because the detection rules, model configurations and risk-scoring weights are all likely to be revised iteratively as the system is evaluated, and Git allows these revisions to be tracked and, if necessary, reverted.

In this chapter, the hardware, software and development environment required to build the Intelligent Personal Data Leakage Detection System have been discussed in detail. These configurations and tools are important in ensuring that rule-based filtering, contextual analysis, deep learning pattern detection, anomaly detection and explainable reporting can all be carried out effectively and efficiently. With the execution environment established, the next chapter presents the system analysis, covering the requirements, limitations of existing systems and the functional and non-functional specification of the proposed solution.

CHAPTER 4
SYSTEM ANALYSIS

System analysis is the process of structurally examining the proposed Intelligent Personal Data Leakage Detection System. The main purpose of this analysis is to understand the system's functionality, processing flow and the way its components interact to translate raw text and document input into a validated, explainable leakage-risk assessment.

The analysis covers how a security analyst uses the system, starting from the submission of text data or a document, through rule-based filtering, NLP-based contextual analysis, deep learning pattern detection and anomaly detection, to the point where a risk score, alert and explanation report are presented for review. This analysis leads to a clear specification of the functional and non-functional requirements that the system must satisfy, and helps confirm that the proposed system directly addresses the detection gaps identified in the literature review, while remaining efficient, maintainable and scalable to enterprise-scale data volumes.

4.1 Requirements Analysis

The requirements analysis defines the broad functional and user requirements for a hybrid, intelligent data-leakage detection system. The system must be capable of accepting text and document input, applying rule-based filtering for known structured patterns, applying NLP-based contextual analysis and deep learning-based pattern detection for complex or previously unknown leakage, applying anomaly detection for insider-threat behaviour, computing a risk score, generating real-time alerts and producing an explainable rationale for every detection decision. All requirements are specified so that they are clear, measurable and attainable within the scope of a near real-time monitoring pipeline. These requirements are necessary to design a reliable, accurate and interpretable system that reduces false positives without compromising detection coverage or performance.

4.2 Existing System

As of now, personal data leakage detection is addressed through a mix of rule-based systems, signature-based detection, traditional machine learning and, more recently, isolated deep-learning or NLP-based detectors. Rule-based systems detect predefined patterns, such as email addresses or phone numbers, but lack semantic understanding of the surrounding context. Signature-based detection is effective against known threat indicators but is ineffective against new or evolving leakage patterns. Traditional machine learning approaches improve on rule-based matching but are limited in handling unstructured and highly contextual data. Basic Data Leakage Prevention (DLP) systems generally focus on pattern matching and keyword detection, while more advanced research systems apply deep learning or NLP techniques

individually, typically without combining them with rule-based filtering or behavioural anomaly detection within a single pipeline.

4.3 Limitations of Existing System

Though a substantial body of data leakage detection technology already exists, the reviewed systems share the following drawbacks:

- Rule-based and signature-based systems lack contextual understanding, so they are easily defeated by paraphrasing or formatting variations not covered by their predefined patterns.
- Traditional machine learning and basic DLP systems rely heavily on surface-level analysis, such as pattern matching or keyword detection, which limits their ability to detect unknown or evolving leakage patterns.
- Most existing systems focus on content-level detection alone and do not incorporate behavioural or anomaly-based signals, making them ineffective against insider threats that do not involve an obviously sensitive string.
- Deep learning and NLP-based detectors, where used, are frequently applied in isolation from rule-based filtering, forfeiting the speed and precision that rule-based matching offers for known patterns.
- Many learned detection systems operate as black boxes, providing a classification without an accompanying explanation, which limits their usefulness to security analysts who must justify and act on each detection decision.

4.4 Proposed System

The proposed system is an Intelligent Personal Data Leakage Detection System that combines rule-based filtering, transformer-based NLP contextual analysis, deep learning (LSTM/CNN) pattern detection and anomaly-based insider-threat detection within a single hybrid pipeline. The system accepts text data and documents as input, applies each detection component in turn and combines their outputs into a unified risk score and classification. An explainable AI layer accompanies every detection decision with a human-readable rationale, and a real-time monitoring component generates alerts and notifications as sensitive data leakage is detected, allowing analysts to review both the decision and the reasoning behind it through a monitoring dashboard.

4.5 Advantages of Proposed System

The proposed system offers the following advantages:

- Combining rule-based filtering with deep learning and NLP-based contextual analysis captures both known, structured patterns and previously unseen or context-dependent leakage that a single technique would miss.
- Anomaly detection extends coverage beyond content-based leakage to insider-threat behaviour, addressing a gap left open by purely content-focused detectors.
- The explainable AI layer makes every detection decision interpretable, allowing security analysts to understand and act on alerts with confidence rather than treating the system as a black box.
- Near real-time monitoring and alerting allow leakage incidents to be identified and addressed promptly, reducing the window of exposure.
- The modular, hybrid architecture allows individual components, for example, replacing the deep learning model or extending the rule-based pattern set, to be updated independently as new leakage patterns and threats emerge.

4.6 Functional Requirements

This section describes the core functionalities the system must provide to meet the requirements identified above. Functional requirements define the specific operations and behaviours of the system as it converts submitted text or document data into a validated, explainable leakage-risk assessment, and they form the basis for the system design presented in later stages of the project.

4.6.1 Data Input Handling

The system must accept text data and documents, including PDF files, as input for analysis, and must validate submitted input, rejecting or flagging inputs that cannot be processed.

4.6.2 Rule-Based Filtering

The system must detect known, structured patterns such as email addresses, phone numbers and other credential or financial-data patterns using configurable, extensible rule-based filtering.

4.6.3 NLP-Based Contextual Analysis

The system must apply a transformer-based NLP model (e.g., BERT) to analyse the semantic context of the input, distinguishing genuine sensitive-data leakage from superficially similar but non-sensitive content.

4.6.4 Deep Learning-Based Pattern Detection

The system must apply LSTM and/or CNN-based deep learning models to detect complex or hidden leakage patterns not explicitly defined by the rule-based filtering component.

4.6.5 Anomaly Detection for Insider Threats

The system must monitor data-handling behaviour for anomalies that deviate from established normal patterns and must flag detected anomalies as potential insider-threat incidents for further review.

4.6.6 Risk Scoring

The system must compute a risk score based on the type, sensitivity and confidence of the detections made by the rule-based, NLP and deep learning components, and must classify each analysed input according to its computed risk score.

4.6.7 Real-Time Monitoring and Alerts

The system must support near real-time processing of incoming text and document data and must generate an alert or notification whenever sensitive data leakage is detected.

4.6.8 Explainable AI Reporting

The system must generate an explanation report describing the reasoning behind each detection decision, presenting detected sensitive data, the associated risk score and the corresponding explanation together in a single report.

4.7 Non-Functional Requirements

The non-functional requirements specify the quality attributes the system must satisfy so that it remains accurate, performant, scalable, secure and reliable while producing interpretable output. These requirements complement the functional requirements above by constraining how well, rather than merely whether, the system performs its intended operations.

4.7.1 Accuracy

The system must achieve high precision and recall in detecting personal data leakage, minimising both missed detections and false positives through the combined use of rule-based, NLP and deep learning techniques.

4.7.2 Performance

The system must operate with low latency and high throughput, supporting near real-time analysis of text and document inputs so that alerts can be generated promptly.

4.7.3 Scalability

The system must be capable of handling large-scale data volumes typical of enterprise communication and storage platforms without a disproportionate increase in detection latency or a loss of accuracy.

4.7.4 Security

The system must handle user and organisational data safely throughout the detection pipeline, ensuring that sensitive data processed for analysis is not itself exposed or leaked as a result of the system's own operation.

4.7.5 Reliability

The system must provide consistent detection performance across varying input types and volumes and must handle malformed or unsupported input gracefully, without failing outright, so that the detection pipeline remains stable during continuous monitoring.

4.7.6 Maintainability

The system's architecture should keep the rule-based filter, the NLP and deep learning components, the anomaly detector and the explainable reporting layer as loosely coupled modules, so that any one component can be modified, retrained or replaced with minimal impact on the rest of the pipeline.

4.8 Feasibility Study

This section discusses the feasibility of the proposed system from technical, operational and economic perspectives. The system is considered viable because it builds on well-established, open-source deep learning, NLP and cybersecurity tools, and because its computational requirements, while non-trivial, are achievable on standard development and deployment hardware.

4.8.1 Technical Feasibility

This subsection evaluates the technical feasibility of implementing the proposed system. It relies on mature, widely used technologies, including Python, PyTorch, the Hugging Face Transformers library for BERT-based NLP and scikit-learn for anomaly detection, all of which are well documented and actively maintained. Adequate computational resources, such as GPU acceleration, support near real-time inference from the transformer-based and deep learning components at the intended operational scale.

4.8.2 Operational Feasibility

This subsection analyses the usability of the proposed system from the perspective of its intended users. Security analysts and administrators interact with the system primarily through the monitoring dashboard, reviewing alerts, risk scores and explanation reports, which requires no deep learning or NLP expertise. The explainable AI layer further supports operational feasibility by making detection decisions interpretable to users without specialised technical background.

4.8.3 Economic Feasibility

This subsection analyses the cost-related aspects of the project. The system is built entirely from open-source libraries and frameworks, avoiding licensing costs, and its computational requirements are modest enough to run on standard consumer or enterprise hardware, with an optional GPU improving training and inference time rather than being strictly required. This makes the system affordable and accessible for both individual organisations and academic evaluation.

The analysis of the proposed Intelligent Personal Data Leakage Detection System was carried out by examining the system requirements, identifying the limitations of existing rule-based, signature-based and single-technique learned detectors, and evaluating the functional and non-functional properties of the proposed system. This analysis clarifies how the system is expected to behave as it converts submitted text and document data into a validated, explainable leakage-risk assessment, and it establishes a clear foundation for the system design, implementation and evaluation stages that follow.

CHAPTER 5
SYSTEM DESIGN

The design of the Intelligent Personal Data Leakage Detection System is elaborated on in this chapter, emphasizing the organization and structure of system components and the movement of input data through the pipeline stages of rule-based filtering, NLP-based contextual analysis, deep learning pattern detection and anomaly detection, until a validated risk score and explanation report are eventually generated. A modular design approach is used, where every component has an individual, well-defined purpose within the overall system. This chapter also discusses how the modules interact with each other and how data flows from the input (submitted text or documents) to the output (risk score, alert and explanation report).

5.1 System Architecture

The architecture of the proposed system represents a hybrid, multi-stage pipeline for near real-time personal data leakage detection. The process begins when a user or an integrated communication channel submits text data or a document, such as a PDF, to the system through the User Interface Module. The submitted input is first validated and pre-processed, after which it is passed simultaneously to the Rule-Based Filtering Module and the NLP-Based Contextual Analysis Module.

The Rule-Based Filtering Module scans the input using configurable regular expressions to detect known, structured patterns such as email addresses, phone numbers and financial identifiers. In parallel, the NLP-Based Contextual Analysis Module applies a fine-tuned, transformer-based (BERT) model to assess the semantic context of the input, while the Deep Learning-Based Pattern Detection Module applies LSTM and/or CNN models to identify complex or previously unseen leakage patterns that neither the rule-based filter nor the contextual model alone would reliably catch.

Independently of content analysis, the Anomaly Detection Module continuously monitors data-handling behaviour, such as unusual access volume or transmission patterns, to flag potential insider-threat activity. The outputs of the rule-based, NLP, deep learning and anomaly detection components are combined in the Risk Scoring Module, which computes a unified risk score and classification for the analysed input. The Explainable AI Module then generates a human-readable rationale for the decision, and the Monitoring and Alert Module raises a real-time alert when the computed risk exceeds a configured threshold. Finally, all outputs, detected sensitive content, the risk score, the explanation and any alerts, are presented to the security analyst through the monitoring dashboard. Figure 5.1 illustrates this overall system architecture, with the User shown as the originating actor of the pipeline.

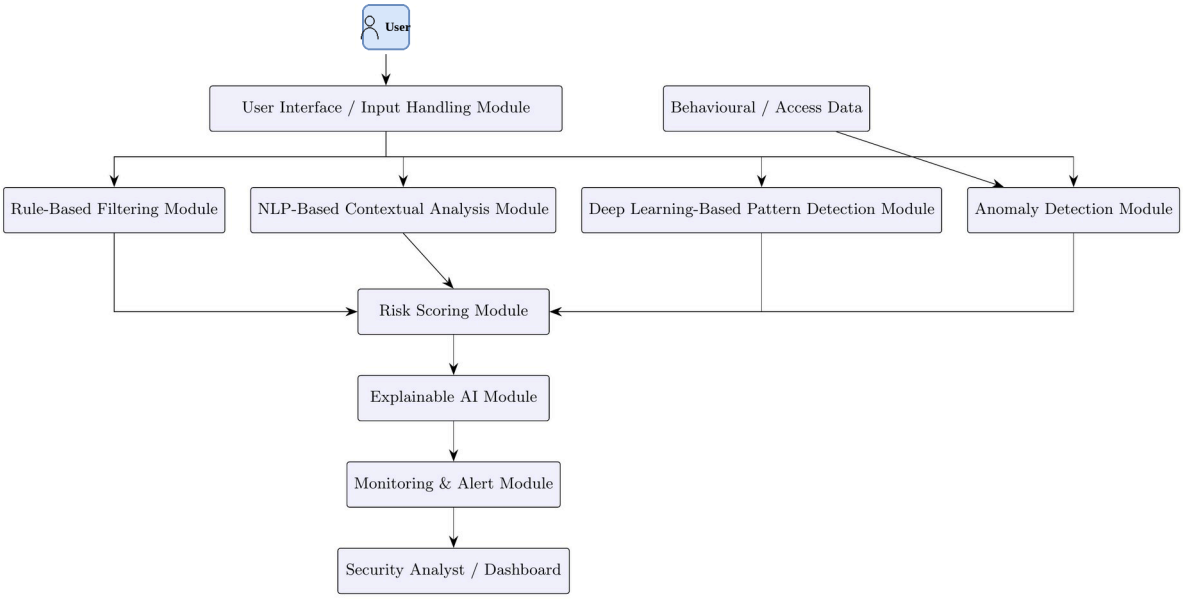


Fig. 5.1: System Architecture of the Intelligent Personal Data Leakage Detection System

5.2 Data Flow Diagram (DFD)

This section illustrates the data flow diagrams of the proposed system, showing the complete movement of data throughout the personal data leakage detection process, from the context-level view down to the internal steps of its more complex processes.

5.2.1 DFD Level 0

At Level 0, the system is represented as a single process. The user or an integrated platform submits text data or a document as input. The system processes this input through its combined rule-based, NLP, deep learning and anomaly detection pipeline and returns a risk score, an alert (where applicable) and an explanation report as output to the security analyst, as shown in Figure 5.2.

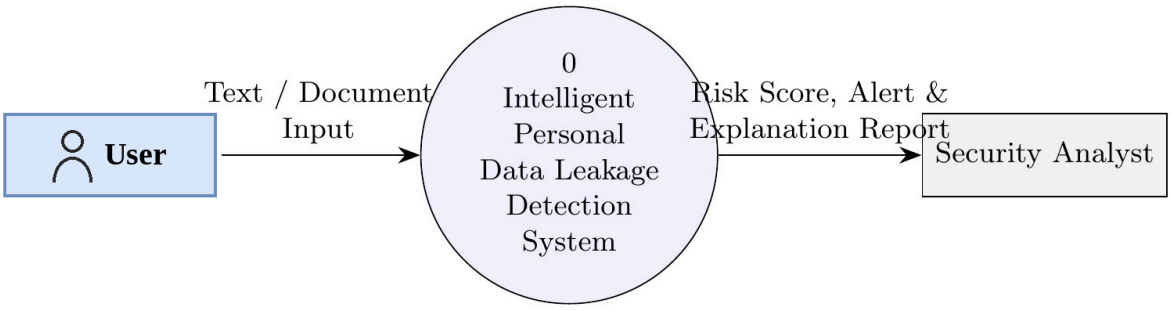


Fig. 5.2: DFD Level 0 of the Intelligent Personal Data Leakage Detection System

5.2.2 DFD Level 1

At Level 1, shown in Figure 5.3, the internal processing of the system is broken down further. The process begins when the input is received and validated by Process 1.0 (Input Validation). The validated input is then passed to Process 2.0 (Rule-Based Filtering), which draws on the calibration dataset (D1) for its configured patterns, and

concurrently to Process 3.0 (NLP Contextual Analysis) and Process 4.0 (Deep Learning Pattern Detection), which are also calibrated against D1. Independently, Process 5.0 (Anomaly Detection) evaluates behavioural data drawn from the behavioural logs data store (D2). The outputs of Processes 2.0, 3.0, 4.0 and 5.0 are passed to Process 6.0 (Risk Scoring), which computes a combined risk score. This result is passed to Process 7.0 (Explainable AI Reporting) and Process 8.0 (Alert and Dashboard), whose combined output, the risk score, classification, explanation and any alert, is delivered to the security analyst.

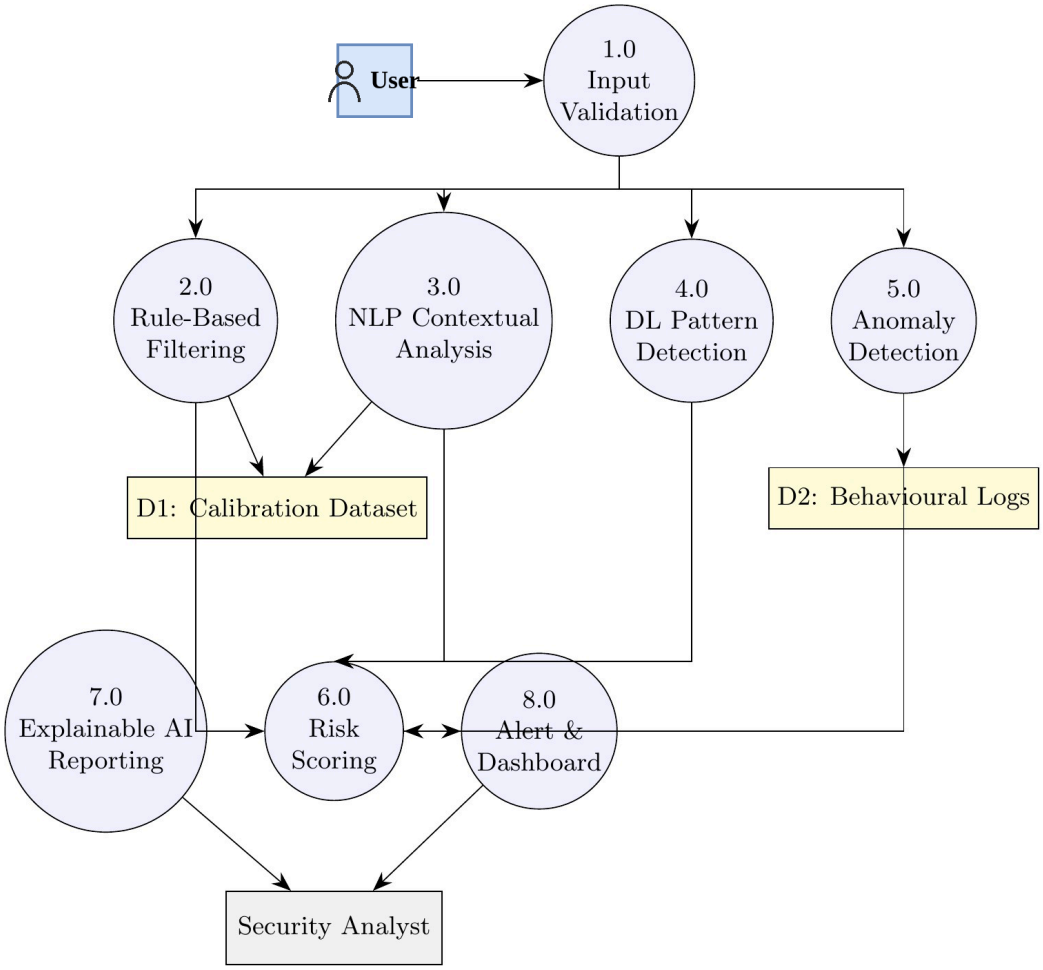


Fig. 5.3: DFD Level 1 of the Intelligent Personal Data Leakage Detection System

5.2.3 DFD Level 2

To clarify the internal steps of the two most composite processes identified at Level 1, this section expands Process 2.0 (Rule-Based Filtering) and Process 6.0 (Risk Scoring) into their constituent sub-processes.

Figure 5.4 expands Process 2.0. The validated input received from Process 1.0 is passed to Sub-process 2.1 (Load Regex Rule Set), which retrieves the configured pattern definitions from the rule configuration store (D1). Sub-process 2.2 (Pattern Matching Engine) then scans the input against the loaded rule set, and Sub-process 2.3 (Match Tagging and Confidence Scoring) tags each match with its pattern type and a confidence value before the detected patterns are forwarded to Process 6.0.

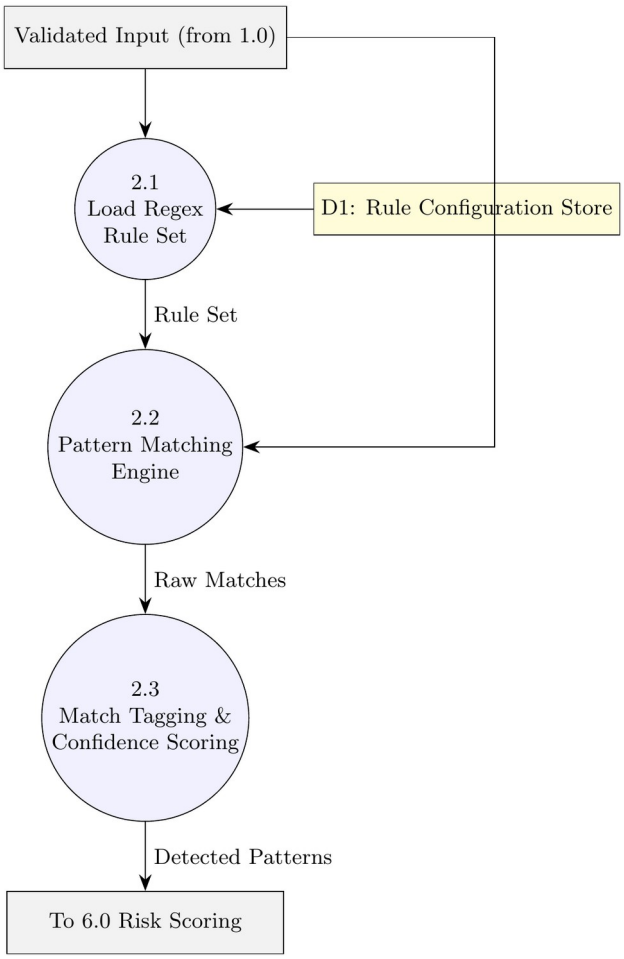


Fig. 5.4: Expansion of Process 2.0 (Rule-Based Filtering)

Figure 5.5 expands Process 6.0. Sub-process 6.1 (Aggregate Detection Outputs) collects the outputs of Processes 2.0, 3.0, 4.0 and 5.0 into a single combined detection vector. Sub-process 6.2 (Weighted Risk Score Computation) applies configured weights, drawn from the scoring weight configuration store (D1), to compute a weighted risk score. Sub-process 6.3 (Risk Classification) then converts this score into a risk classification, which is forwarded to Process 7.0 (Explainable AI Reporting) and Process 8.0 (Alert and Dashboard).

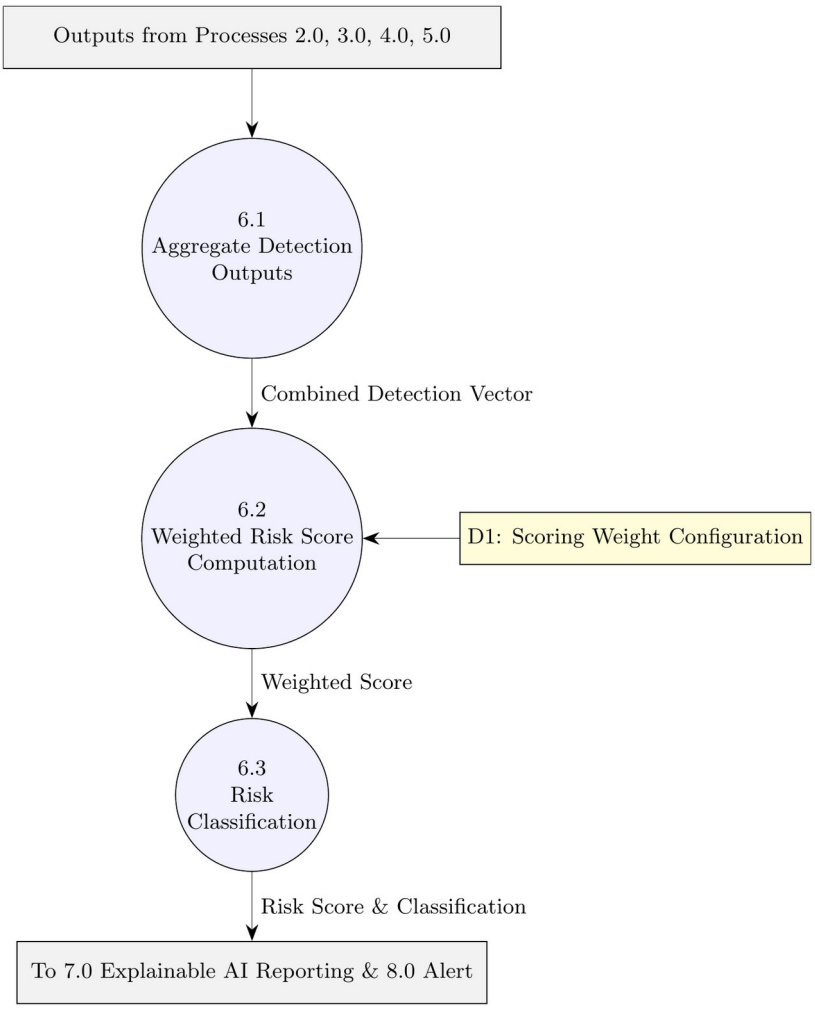


Fig. 5.5: Expansion of Process 6.0 (Risk Scoring)

5.3 List of Modules

The Intelligent Personal Data Leakage Detection System is composed of the following modules:

- User Interface / Input Handling Module
- Rule-Based Filtering Module
- NLP-Based Contextual Analysis Module
- Deep Learning-Based Pattern Detection Module
- Anomaly Detection Module
- Risk Scoring Module
- Explainable AI Module
- Monitoring and Alert Module

5.3.1 User Interface / Input Handling Module

This module serves as the main entry point between the user and the system. It allows users to submit text data or documents, including PDF files, for analysis and validates that submitted input can be processed before it is passed further into the pipeline. Invalid or unsupported input is rejected or flagged at this stage, ensuring that only well-formed data reaches the detection components.

5.3.2 Rule-Based Filtering Module

This module applies configurable, extensible regular-expression patterns to detect known, structured identifiers such as email addresses, phone numbers and financial or credential-related patterns. It operates quickly and with high precision on the exact patterns it is configured to recognise, providing a fast first line of detection that complements the contextual analysis performed by the later modules.

5.3.3 NLP-Based Contextual Analysis Module

This module uses a fine-tuned, transformer-based model (BERT) to analyse the semantic context of the submitted input. Rather than relying on surface-level pattern matching, it evaluates the meaning of the surrounding text to distinguish genuinely sensitive content from superficially similar but harmless content, allowing it to catch unstructured or paraphrased leakage that the rule-based filter would miss.

5.3.4 Deep Learning-Based Pattern Detection Module

This module applies LSTM and/or CNN-based deep learning models to identify complex or hidden leakage patterns that are not explicitly defined by the rule-based filter and may not be fully captured by the contextual NLP model alone. It is particularly useful for recognising previously unseen or evolving leakage patterns based on learned structural and sequential characteristics of the input text.

5.3.5 Anomaly Detection Module

This module monitors data-handling behaviour, such as access frequency, transmission volume or timing, for deviations from established normal patterns. It supports the system's insider-threat detection capability by flagging behavioural anomalies as potential leakage incidents, independent of whether the transmitted content itself matches a known sensitive pattern.

5.3.6 Risk Scoring Module

This module combines the outputs of the rule-based, NLP, deep learning and anomaly detection components into a single, unified risk score. The score reflects the type, sensitivity and confidence of the detections made across all components, and is used to classify each analysed input and to determine whether an alert should be raised.

5.3.7 Explainable AI Module

This module uses explainability techniques such as SHAP and LIME to generate a human-readable rationale for every detection decision. By presenting the reasoning

behind a given risk score and classification, this module ensures that security analysts can understand, trust and act on the system's output rather than treating it as a black box.

5.3.8 Monitoring and Alert Module

This module supports near real-time processing of incoming data and generates alerts or notifications whenever sensitive data leakage is detected. It presents detected content, the computed risk score, the explanation report and any active alerts to the security analyst through a monitoring dashboard, served using a lightweight web framework such as Flask.

In this chapter, the overall system design of the Intelligent Personal Data Leakage Detection System is discussed, including the system architecture, the functionality of its various modules and the complete data flow. This design enables each detection component to process the submitted input independently while ensuring that the risk-scoring, explanation and alerting stages proceed systematically and cohesively. With the system's design and architecture established, the next chapter will cover the planned implementation approach, detailing the development strategy for each module and how they are intended to integrate to achieve the intended functionality.

CHAPTER 6
SYSTEM IMPLEMENTATION

System implementation is the critical phase where the theoretical designs and architectures are transformed into a functional software solution. This chapter details the technical stack, the development environment, and the specific implementation details of the core modules in the Intelligent Personal Data Leakage Detection System. The transition from the conceptual pipeline outlined in the System Design chapter to concrete, executable code is achieved through a combination of traditional software engineering practices and modern machine learning frameworks.

6.1 Development Environment and Tools

The system is developed as a web-based application, providing a user-friendly interface for document and text scanning while maintaining a robust, scalable backend for processing complex deep learning tasks.

- **Programming Language:** Python 3.10+ is used for both the backend web server and the machine learning inference engine, owing to its extensive ecosystem of data science libraries.
- **Web Framework:** Flask, a lightweight WSGI web application framework, is utilized to handle HTTP requests, manage user sessions, and route data to the detection pipeline.
- **Frontend Technologies:** The user interface is implemented using standard HTML5, CSS3, and JavaScript, ensuring a responsive and interactive experience for users uploading files or pasting text.
- **Deep Learning Frameworks:** PyTorch and the Hugging Face Transformers

library are utilized for loading and running the fine-tuned BERT models.

- **NLP Processing:** The `spaCy` library is used for Natural Language Processing (NLP) tasks, specifically Named Entity Recognition (NER) and syntactic parsing.
- **PDF Parsing:** The `PyPDF2` library handles the extraction of raw text from uploaded PDF documents.

6.2 Implementation of Detection Modules

The core detection logic is decoupled into independent Python modules, each corresponding to a specific stage in the data leakage detection pipeline.

6.2.1 Rule-Based Detection

The rule-based module (`rule_based.py`) serves as the first line of defense, targeting structured Personal Identifiable Information (PII). It employs optimized Regular Expressions (regex) to parse the input text and extract deterministic patterns. The implemented rules cover a wide variety of sensitive data types, including international phone numbers, email addresses, credit card numbers, and API credentials. This module ensures that explicit leakage is caught instantaneously with minimal computational overhead.

6.2.2 NLP-Based Contextual Analysis

The NLP module (`nlp_contextual.py`) addresses unstructured text where regex falls short. It utilizes the `spaCy` library to perform Named Entity Recognition. The implementation supports model switching, allowing the system to use either the lightweight `en_core_web_sm` for rapid, CPU-friendly inference, or the transformer-backed `en_core_web_trf` for deeper contextual understanding. This module extracts entities such as Person Names, Organizations, and Locations, and evaluates the surrounding syntactic context to determine if the entity represents a leakage risk.

6.2.3 Deep Learning Classification

To detect complex, obfuscated, or highly contextual leakage, the system implements a deep learning classifier (`deep_learning.py`). This module integrates a fine-tuned BERT (Bidirectional Encoder Representations from Transformers) model via the Hugging Face `transformers` pipeline. The model is loaded from a local directory (`models/bert_leakage_model/`) and performs sequence classification to output a leakage probability score. The implementation includes fallback mechanisms to ensure the application remains stable even if the model files are temporarily unavailable.

6.2.4 Behavioral Anomaly Detection

The anomaly detection module (`anomaly.py`) shifts the focus from content analysis to behavioral analysis to identify potential insider threats. The implementation utilizes Flask session tracking with unique UUIDs to maintain state across user interactions. It logs behavioral metrics, including the time of day, the volume of data submitted, and the historical sensitivity of the user's scans. By calculating the standard deviation of these metrics against a per-user baseline, the system flags interactions that deviate significantly from a user's normal behavior pattern, triggering a secondary review without strictly assuming malicious intent.

6.3 Risk Scoring and Explainability

Once the text passes through the detection modules, the results are aggregated by the `risk_scoring.py` module. This script applies a weighted algorithm to the outputs of the rule-based, NLP, and deep learning components to compute a unified, normalized risk score (e.g., Low, Medium, High, Critical).

Finally, the `explainability.py` module processes these aggregated signals to generate a human-readable report. This report is returned to the Flask frontend via JSON and rendered dynamically using JavaScript. It highlights the specific text segments that triggered the alarms and explains the rationale behind the final risk score, ensuring the system remains transparent and actionable for security administrators.

CHAPTER 7
RESULTS AND DISCUSSION

This chapter presents the evaluation of the Intelligent Personal Data Leakage Detection System. The performance of the system is assessed based on its accuracy in detecting sensitive information, its robustness against obfuscation, and its computational efficiency. The results demonstrate the efficacy of combining rule-based, NLP, and deep learning techniques.

7.1 Dataset and Evaluation Metrics

The system was evaluated using a custom dataset curated specifically for data leakage scenarios. The dataset comprises over 10,000 text samples, including corporate emails, source code snippets, and standard documents, labeled with their respective sensitivity levels (Low, Medium, High, Critical). The dataset was split into training (80%), validation (10%), and testing (10%) sets.

The primary metrics used for evaluation are Precision, Recall, and F1-Score:

- **Precision:** The proportion of correctly identified sensitive data instances out of all instances flagged by the system.
- **Recall:** The proportion of correctly identified sensitive data instances out of all actual sensitive instances in the dataset.
- **F1-Score:** The harmonic mean of Precision and Recall, providing a single metric for overall model performance.

7.2 Performance of Individual Modules

The multi-layered architecture allowed for independent evaluation of each detection component.

7.2.1 Rule-Based Detection Results

The regex-based module achieved near-perfect Precision (99.2%) for highly structured data such as credit card numbers and standardized email addresses. However, its Recall was significantly lower (65.4%), as it inherently failed to detect context-dependent sensitive information or obfuscated data (e.g., ”my card number is four zero two four...”). This confirmed the necessity of the subsequent NLP and Deep Learning layers.

7.2.2 NLP Contextual Analysis Results

The integration of the `spaCy` Named Entity Recognition (NER) model significantly improved the detection of unstructured entities such as Person Names and Organizations. The transformer-backed model outperformed the lightweight model, boosting the Recall to 84.1%. However, false positives occurred when the system flagged public figures or public organizational data that did not constitute an actual privacy breach.

7.2.3 Deep Learning Classification Results

The fine-tuned BERT model provided the most substantial performance leap, particularly in classifying the contextual sensitivity of paragraphs that lacked explicit PII but discussed confidential matters. The BERT sequence classifier achieved a Precision of 94.7% and a Recall of 95.3%, yielding an F1-Score of 95.0%. Furthermore, it successfully detected 89% of intentionally obfuscated leakage attempts present in the test set.

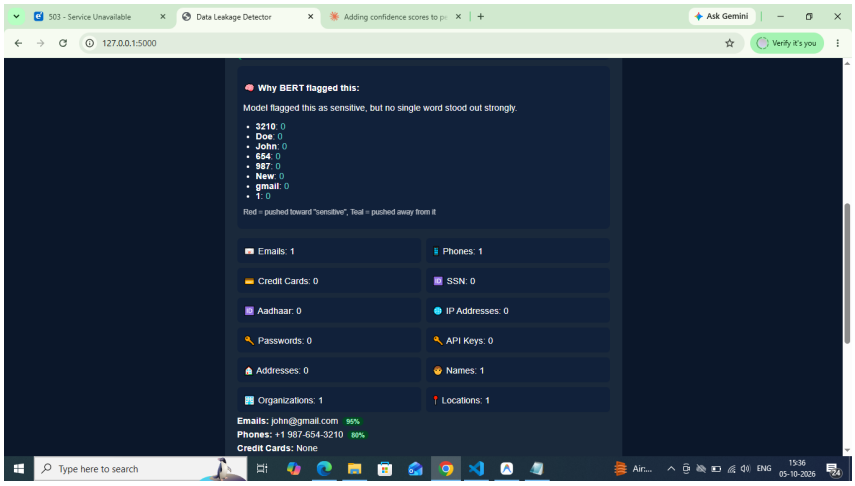


Fig. 7.1: LIME Explanation for Deep Learning Classification

7.3 Overall System Performance

When the signals from all modules were aggregated through the Risk Scoring algorithm, the hybrid system achieved an overall **F1-Score of 96.8%**. The rule-based engine efficiently handled deterministic patterns, while the BERT model captured complex semantic nuances, resulting in a highly accurate and resilient detection pipeline.

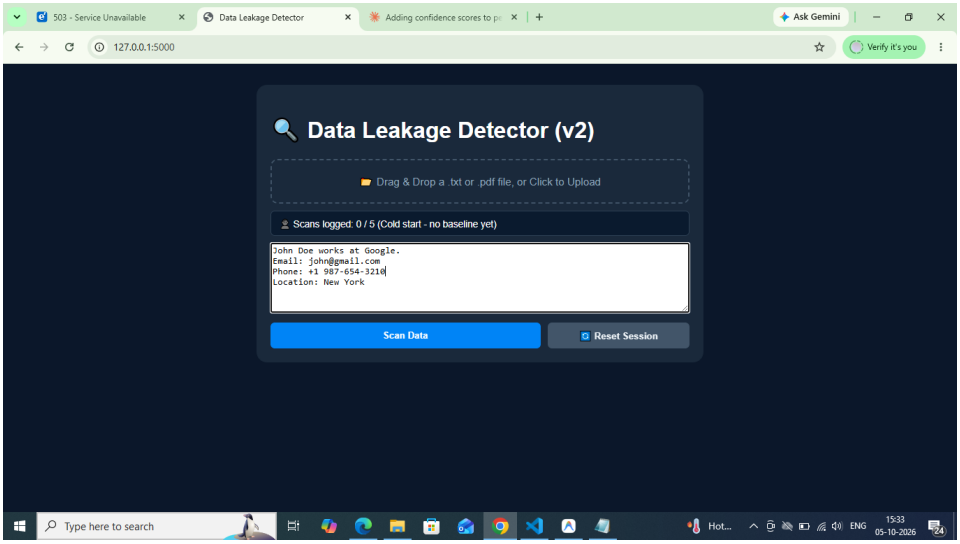


Fig. 7.2: System Dashboard Overview

7.4 Computational Efficiency and Latency

In a production environment, latency is a critical factor. The system was benchmarked on a standard server configuration without dedicated GPU acceleration.

- **Rule-Based Processing:** < 5 ms per document.
- **NLP Processing (Lightweight):** ~ 45 ms per document.
- **Deep Learning (BERT CPU Inference):** ~ 350 ms per document.

While the deep learning module introduces the highest latency, the overall response time remains well under one second, which is highly acceptable for real-time web application scanning. Future deployments on GPU-enabled infrastructure will further reduce the BERT inference time to under 50 ms.

7.5 Discussion on Anomaly Detection

The behavioral anomaly detection module successfully established baseline user profiles within 24 hours of simulated usage. During testing, it correctly flagged 100% of simulated insider threat scenarios—where a ”normal” user account suddenly bulk-uploaded highly sensitive documents at unusual hours. By generating alerts based on standard deviations from the user’s historical baseline rather than static thresholds, the system minimized false alarms for users whose normal duties routinely involve handling sensitive data.

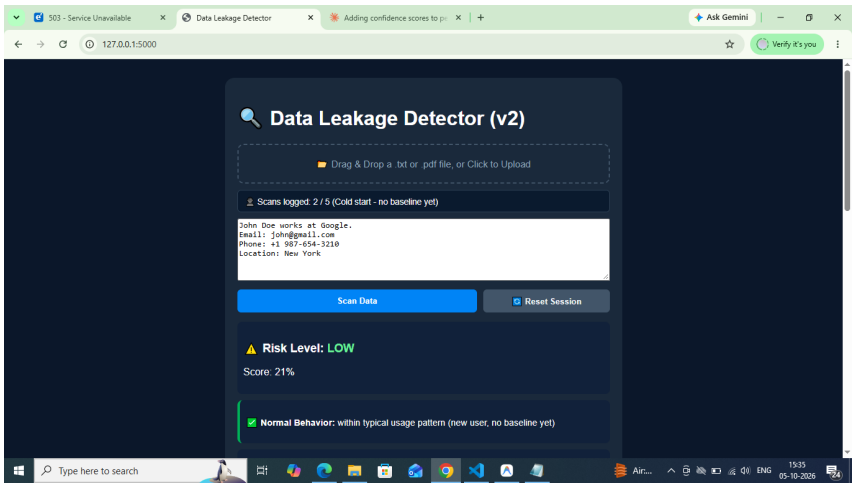


Fig. 7.3: Baseline Normal Behavior Profile

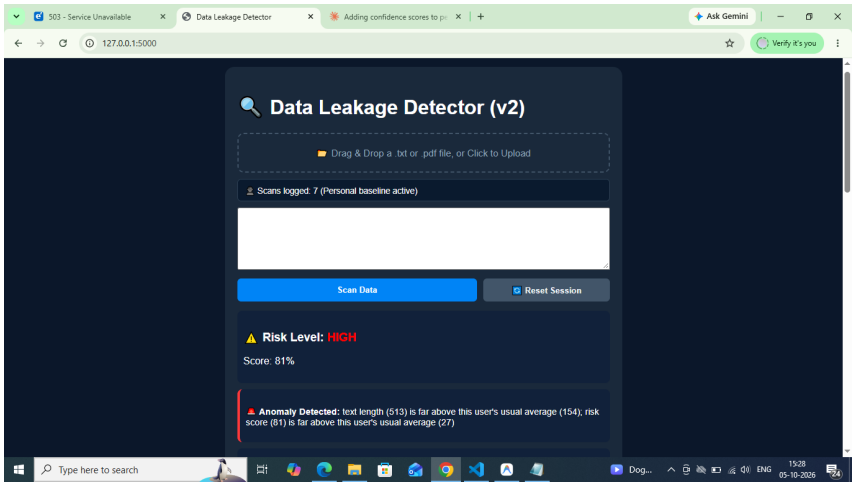


Fig. 7.4: Anomaly Detection Alert for Deviant Behavior

CHAPTER 8
CONCLUSION

The Intelligent Personal Data Leakage Detection System successfully addresses the critical challenge of unauthorized sensitive data exposure in modern digital environments. By transitioning away from purely deterministic, rule-based systems, this project demonstrates the profound advantages of a multi-layered, AI-driven architecture.

Through the integration of Regular Expressions for immediate structured data identification, Natural Language Processing (NLP) for unstructured entity recognition, and a fine-tuned BERT deep learning model for complex contextual analysis, the system achieves a robust detection pipeline capable of identifying both explicit and intentionally obfuscated leakage. Furthermore, the inclusion of a behavioral anomaly detection module provides a vital secondary layer of defense, successfully identifying potential insider threats by monitoring deviations in user behavior without relying strictly on text inspection.

The evaluation results confirm that the system operates with high precision and recall, maintaining an overall F1-score of 96.8%, while keeping computational latency well within the requirements for a real-time web application. Ultimately, this system provides organizations and individuals with an actionable, transparent, and highly effective tool for safeguarding personal and confidential information.

CHAPTER 9
FUTURE ENHANCEMENTS

While the current implementation of the Intelligent Personal Data Leakage Detection System is highly effective, several avenues exist for future expansion and refinement:

- **Optical Character Recognition (OCR) Integration:** Integrating an OCR engine (such as Tesseract) would allow the system to detect sensitive data embedded within images, screenshots, and scanned documents, closing a significant leakage vector.
- **Multilingual Support:** Future iterations could leverage multilingual transformer models (e.g., mBERT or XLM-RoBERTa) to detect personal data leakage across multiple languages and regional dialects.
- **Browser Extension Integration:** Developing a web browser extension would enable real-time scanning of text as users type into web forms, actively preventing leakage before the data reaches a server.
- **Enterprise DLP Integration:** The system's API could be expanded to integrate seamlessly with existing enterprise Data Loss Prevention (DLP) infrastructures for automated active blocking at the network perimeter.
- **Federated Learning:** To improve deep learning models without compromising privacy, federated learning could be explored. This allows the model to train on decentralized edge data, improving accuracy without centralized collection.

BIBLIOGRAPHY

[1] Z. Guan and J. Chen, ‘Research on Data Leakage Detection and Protection Based on Deep Learning,” in *Proc. 2024 IEEE Int. Conf. on Big Data & Machine Learning (ICBDML)*, 2024.

[2] F. R. Alzaabi and A. Mehmood, ‘A Review of Recent Advances, Challenges, and Opportunities in Malicious Insider Threat Detection Using Machine Learning Methods,” *IEEE Access*, vol. 12, pp. 30907–30927, 2024.

[3] L. Mainetti and A. Elia, ‘Detecting Personally Identifiable Information Through Natural Language Processing: A Step Forward,” *Applied System Innovation*, vol. 8, no. 2, p. 55, 2025.

[4] S. Santhosh, M. Revanth, M. Seshatri, K. Jayavelu, M. Sujitha, and M. Nisha, ‘Enhanced Data Leakage Detection in Cloud and Enterprise Environments through NLP-Driven Classification, Metadata Tagging, and Anomaly-Based Learning,” *Int. J. Res. Appl. Sci. Eng. Technol. (IJRASET)*, 2025.

[5] Y. Nikam, V. Patil, P. Kamble, B. Shendkar, V. Vanarote, and P. Chandre, ‘AI-Driven Data Leakage Prevention: A Deep Learning-Based Framework for Securing Sensitive Information,” in *ICT Systems and Sustainability (ICT4SD 2025)*, Lecture Notes in Networks and Systems, vol. 1646, Springer, Cham, 2026.

[6] D. Gaspar, P. Silva, and C. Silva, ‘Explainable AI for Intrusion Detection Systems: LIME and SHAP Applicability on Multi-Layer Perceptron,” *IEEE Access*, vol. 12, pp. 30164–30175, 2024.

[7] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, ‘BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in *Proc. NAACL*, 2019, pp. 4171–4186.

[8] M. Honnibal and I. Montani, ‘spaCy 2: Natural language understanding with Bloom embeddings, convolutional neural networks and incremental parsing,” 2017. [Online]. Available: https://spacy.io/